

Advanced MVC Concepts & UI Enhancements

Day 3



Agenda for Day 3

- Reusable UI Components: Partial Views
- Reusable UI Components: View Components
- Tag Helpers
- Layouts and Sections (Review)
- Client-side Validation
- AJAX with MVC (jQuery.ajax, Fetch API)
- Hands-on Demo & Lab Assignment

Partial Views

- **Purpose:** Render reusable chunks of Razor markup.
- **When to use:** Simple, static UI elements, or rendering a sub-model.
- **Location:** Typically in `Views/Shared/` or `Views/[ControllerName]/`.
- **Rendering:**
 - `@Html.Partial("_PartialName", model)` (older, synchronous)
 - `<partial name="_PartialName" model="model" />`
 - (Tag Helper, asynchronous, recommended)
- **Data Passing:** Can pass a `model`, `ViewData`, or `ViewBag`.

Partial View Example

```
public class Product
{
    public int Id { get; set; }
    public string Name { get; set; }
    public decimal Price { get; set; }
}
```

Partial View Example

```
<!-- _ProductCard.cshtml -->
<div class="card">
  <h5>@Model.Name</h5>
  <p>Price: @Model.Price.ToString("C")</p>
</div>

<!-- In Index.cshtml -->
@foreach (var item in Model)
{
  <partial name="_ProductCard" model="item" />
}
```

View Components

- **Purpose:** More powerful and testable alternative to Partial Views for complex, reusable UI logic.
- **When to use:** Dynamic UI elements that require business logic, database access, or dependency injection.
- **Structure:**
 - A C# class inheriting from **ViewComponent** (e.g., `ProductSummaryViewComponent.cs`).
 - Must have an **InvokeAsync()** method.
 - A default view (`Default.cshtml`) in `Views/Shared/Components/[ComponentName]/`.
- **Rendering:** `@await Component.InvokeAsync("ComponentName", [parameters])`

View Component Example

```
// ViewComponents/ProductSummaryViewComponent.cs
public class ProductSummaryViewComponent : ViewComponent
{
    private readonly ApplicationDbContext _context;
    public ProductSummaryViewComponent(ApplicationDbContext context)
    { _context = context; }
    public async Task<IViewComponentResult> InvokeAsync()
    {
        var totalProducts = await _context.Products.CountAsync();
        ViewBag.TotalProducts = totalProducts;
        return View();
    }
    // Renders /view/shared/ProductSummary/Default.cshtml
}
```

View Component Example

```
//Views/Shared/Components/ProductSummary/Default.cshtml
<div class="alert alert-info">Total Products: @ViewBag.TotalProducts</div>

// In _Layout.cshtml or any View
@await Component.InvokeAsync("ProductSummary")
```


Tag Helpers

- **Purpose:** Server-side code that participates in creating and rendering HTML elements in Razor files.
- **Syntax:** Look like HTML attributes, prefixed with asp-.
- **Benefits:**
 - HTML-friendly development experience.
 - Improved readability of Razor code.
 - Reduced boilerplate code.

Tag Helpers

■ Common Built-in Tag Helpers:

- Form: `asp-action`, `asp-controller`, `asp-route-id`
- Input: `asp-for`
- Validation: `asp-validation-for`, `asp-validation-summary`
- Anchor: `asp-area`, `asp-controller`, `asp-action` → ``Html.ActionLink``
- Environment: `<environment include="Development">`

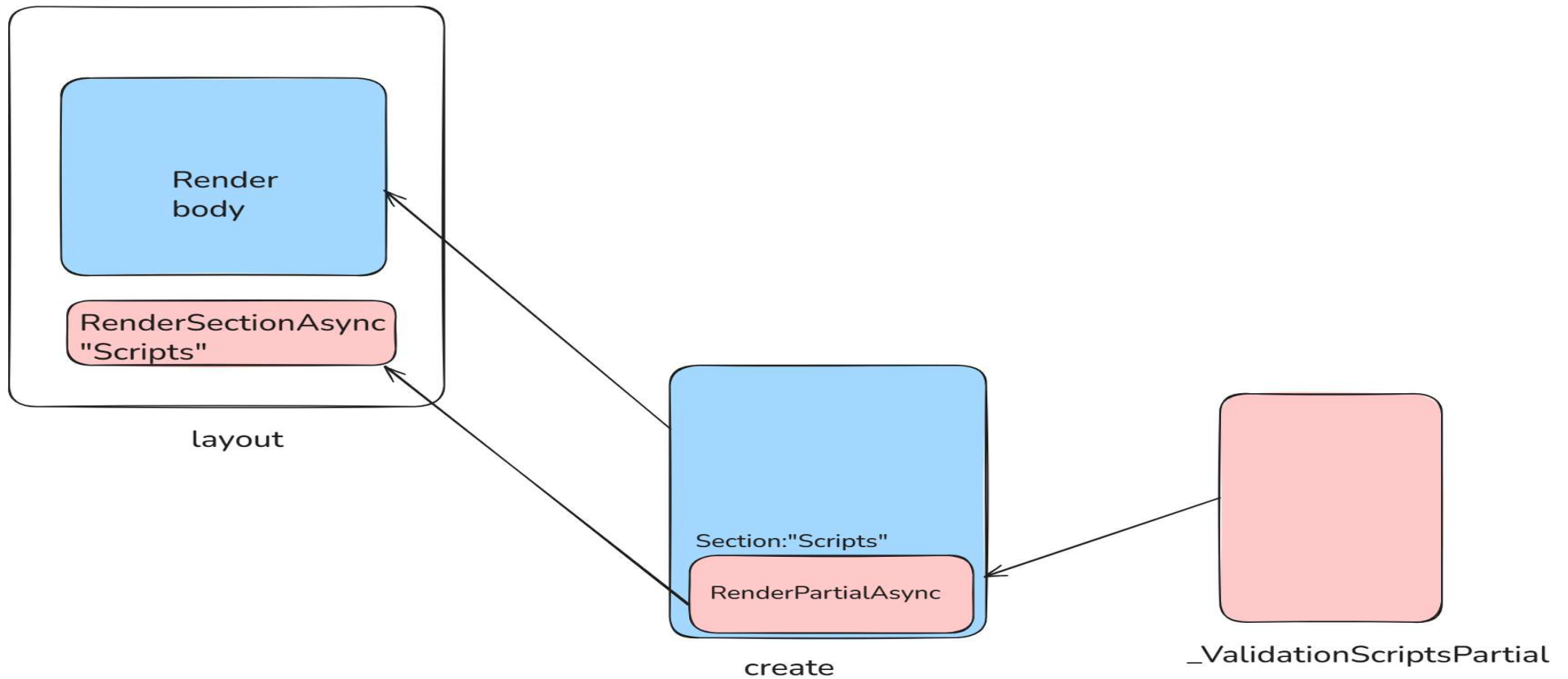
Layouts and Sections (Review)

- **Layouts (*_Layout.cshtml*)**: Provide a consistent look and feel across all pages.
 - * Contains common HTML structure, navigation, footer, CSS, and JavaScript references.
 - **@RenderBody()**: Where the content of the individual view is rendered.
 - **@RenderSection("SectionName", required: false)**: Defines optional content areas.
- ***_ViewStart.cshtml***: Specifies the default layout for all views in a folder and its subfolders.
- ***_ViewImports.cshtml***: Imports common namespaces and Tag Helpers, making them available to all views without explicit **@using** or **@addTagHelper**.

Client-side Validation

- **Purpose:** Provide immediate feedback to the user, reduce server round-trips.
- **How it works:**
 - ASP.NET Core generates HTML5 data-val-* attributes based on Data Annotations.
 - jQuery Validation and jQuery Unobtrusive Validation libraries read these attributes.
- **Setup:** Ensure `_ValidationScriptsPartial.cshtml` is rendered in views with forms (typically via `@section Scripts`).
- **Benefits:** Improved user experience, reduced server load.

Client-side Validation



AJAX (Asynchronous JavaScript and XML) with MVC

- **Purpose:** Update parts of a web page without reloading the entire page.
- **How it works:** JavaScript makes an HTTP request to the server, receives data (often JSON or HTML snippets), and updates the DOM.
- **Controller Actions for AJAX:****
 - Return `JsonResult` for data (e.g., `return Json(products);`).
 - Return `PartialViewResult` for HTML snippets (e.g., `return PartialView("_ProductTablePartial", products);`).

Making AJAX Calls in JavaScript

■ jQuery.ajax():

Client (e.g., Web Browser, Mobile App) <---HTTP Request---> Web API Server
Client sends: GET /api/products Web API Server processes request,
fetches data from database, and sends back: HTTP/1.1 200 OK
Content-Type: application/json [{ "id": 1,
"name": "Laptop", "price": 1200.00 }, { "id": 2,
"name": "Mouse", "price": 25.00 }]

■ fetch API (Modern JavaScript):

```
fetch('/Product/GetProductsJson')  
.then((response => response.json()))  
.then(data => { /* ... update UI ... */ })  
.catch(error => console.error('Error:', error));
```

Assignment

- Create MovieCard partial view and use in index.cshtml of Movies
- Create MovieStatsViewComponent
 - that contain
 - number of movies
 - Average release year
 - Create its default.cshtml
 - Add component to Layout.cshtml (before RenderBody)
- Create **TestJson** Action in Movie Controller and its view



Assignment

- Within its view call `fetch('/Product/GetProductsJson')`
- Display results as text



Assignment

```
document.getElementById('loadJsonBtn').addEventListener('click', () => {
    fetch('/Product/GetProductsJson')
        .then(response => {
            if (!response.ok) {
                throw new Error('Network response was not ok');
            }
            return response.json();
        })
        .then(data => {
            const formattedJson = JSON.stringify(data, null, 2);
            document.getElementById('jsonContainer').textContent = formattedJson;
        })
        .catch(error => {
            console.error('Error loading JSON:', error);
            document.getElementById('jsonContainer').innerHTML =
                '<div class="error">Error loading data: ${error.message}</div>';
        });
});
```